Introduction

The aim of this assignment is to find answers to questions on improve the validation and the test scores of the neural network performing the pneumonia classification of chest x ray images. I ran the classification file a few times while I was setting up and evaluating the code, so the pretrained was only changed to false and froze after a few runs in train mode. I then ran a further baseline run with the model frozen so to have a clean baseline model to start from.

A baseline pneumonia classification model was first run using the provided architecture. This gave training accuracy results of approx. 87.1% and a validation accuracy of 74.8%. These results provided a reference point for later experiments involving transfer learning, augmentation, and other model improvements.

The aim of this assignment was to see why CNN's (Convolutional Neural Networks) are suitable for image classification and to improve the overall model performance and reduce overfitting.

Data Description

The dataset used in this assignment was a trained collection of chest xray images grouped into three categories :

Bacterial

Normal

Viral

The data was stored locally and loaded from separate training and test directories. The assignment brief gave instructions on the initial set up of the code.

```
Found 5419 files belonging to 3 classes.
Using 4336 files for training.
Using 1083 files for validation.
Found 437 files belonging to 3 classes.
Class Names: ['BACTERIAL', 'NORMAL', 'VIRAL']
```

Figure 1 illustrates the features of the chest Xray data.

As you can see on loading there were 5419 files and the split into training and validation plus 437 image files belonging to the three classes above.

These are how the images were visualised in the notebook.

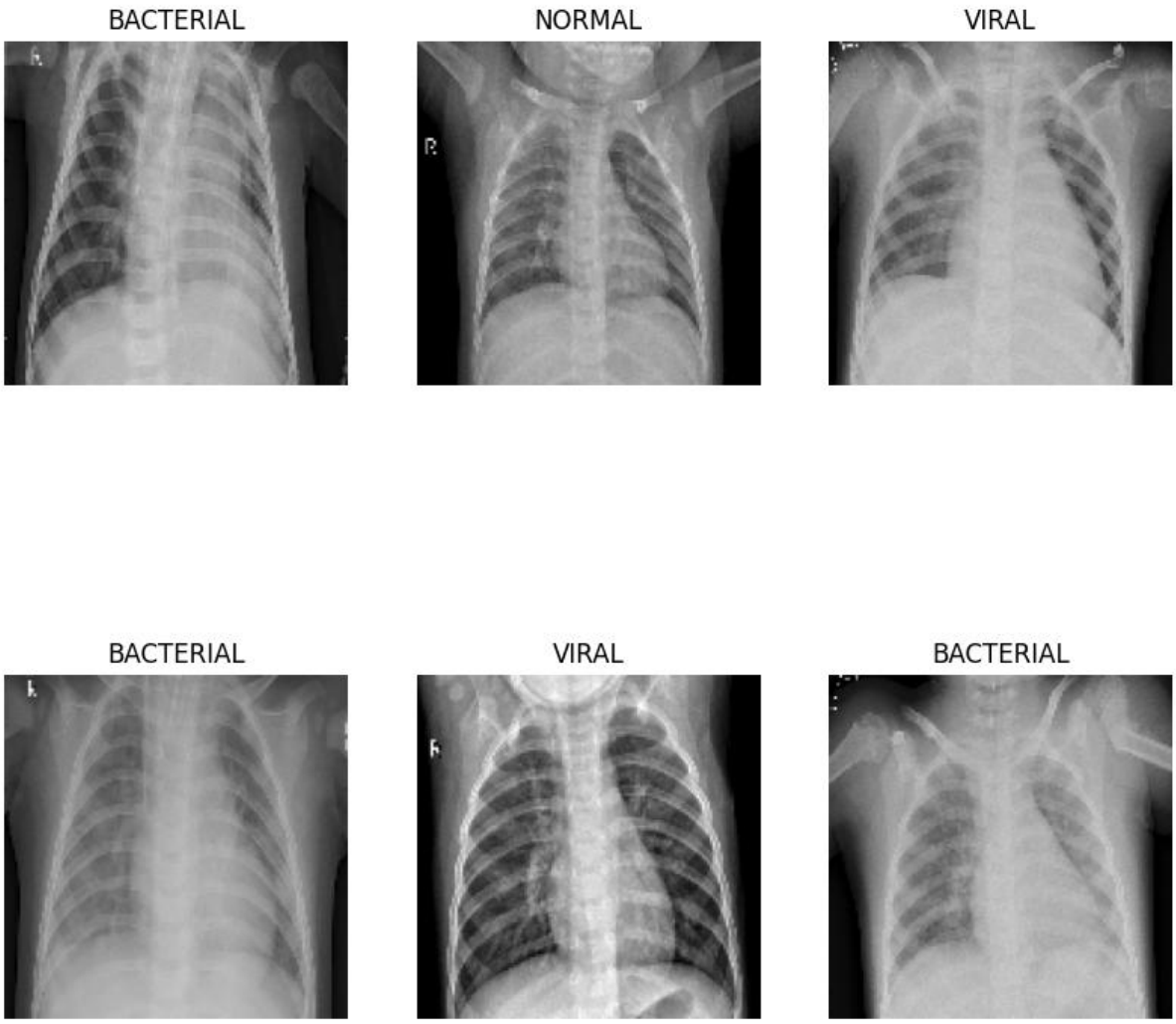

Figure 2 shows the x-ray images of the three classes.

Baseline CNN Model

A baseline CNN model was used as the starting point for the experiments. The model consisted of image rescaling, several convolution and max pooling layers, followed by a flattening stage, a dense hidden layer, dropout, and a final softmax output layer for three classes prediction.

```
model = tf.keras.models.Sequential([
    Input(shape=(img_height, img_width, img_channels)),
    Rescaling(1.0/255),
    Conv2D(16, (3,3), activation='relu'),
    MaxPooling2D(2,2),
    Conv2D(32, (3,3), activation='relu'),
    MaxPooling2D(2,2),
    Conv2D(32, (3,3), activation='relu'),
    MaxPooling2D(2,2),
    Flatten(),
    Dense(512, activation='relu'),
    Dropout(0.2),
    Dense(num_classes, activation='softmax')
])

model.compile(loss='sparse_categorical_crossentropy', optimizer=Adam(), metrics
```

Figure 3 Illustrates the baseline model.

This baseline model provided a reference point for the later experiments. Its results were frozen after the loading and used to identify overfitting and to help improve the model.

Baseline Results

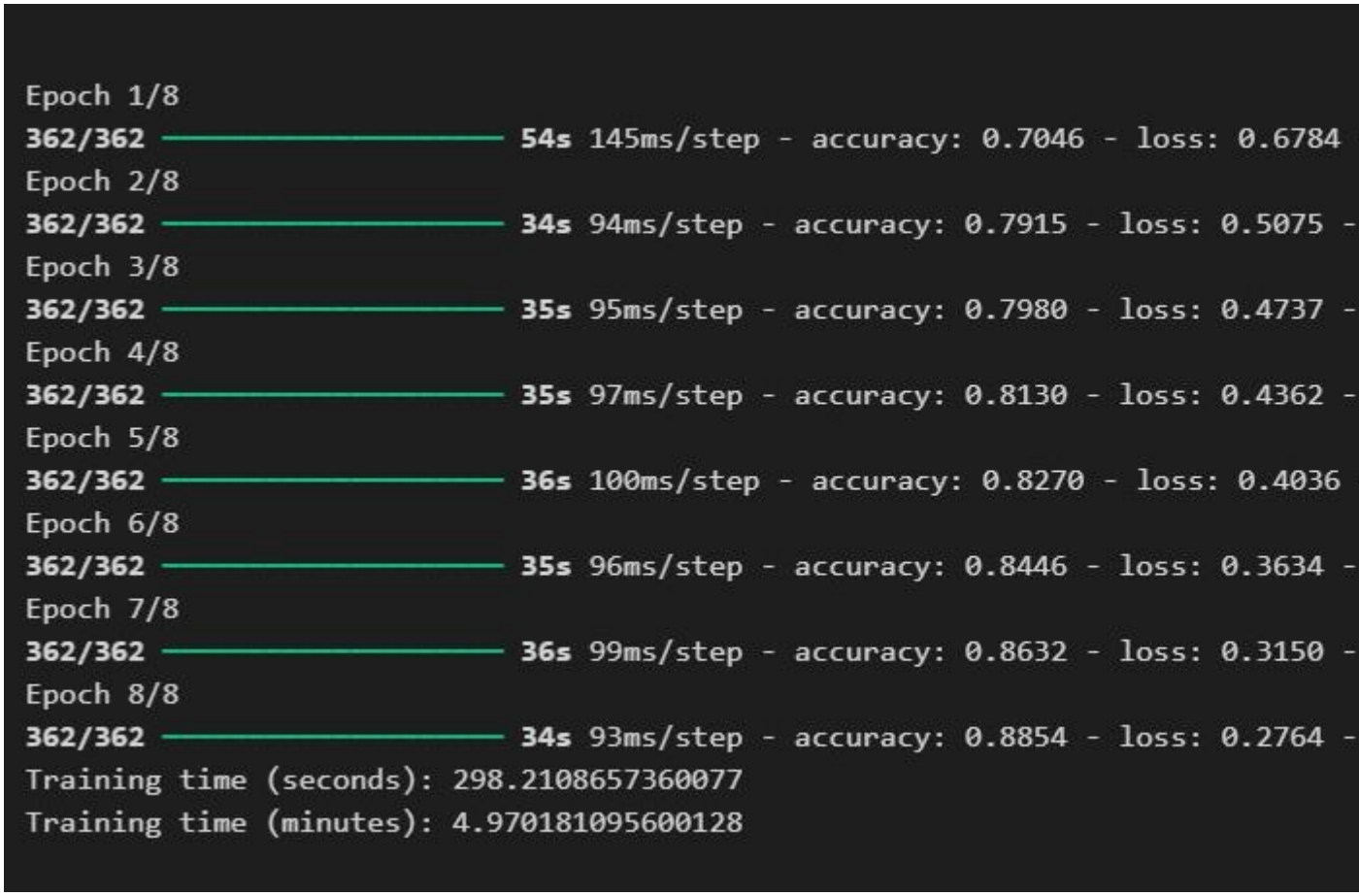

Figure 4 Illustrates the results of the baseline model.

Experiment 1: Global Average Pooling 2D

The first experiment replaced the Flatten() layer, with the GlobalAveragePooling2D(). The reason for this change was to reduce the number of parameters in the model and improve generalisation.

The results showed that this experiment reduced overfitting. The gap between training and validation performance was smaller than in the baseline model, suggesting that the model generalised better to unseen data. However, the final accuracy achieved by this experiment was lower than the baseline model.

This indicated that while Global Average Pooling 2D helped reduce overfitting, it also reduced the model's capacity to achieve the same level of classification accuracy. Therefore, this experiment demonstrated a trade-off between improved generalisation and lower overall performance.

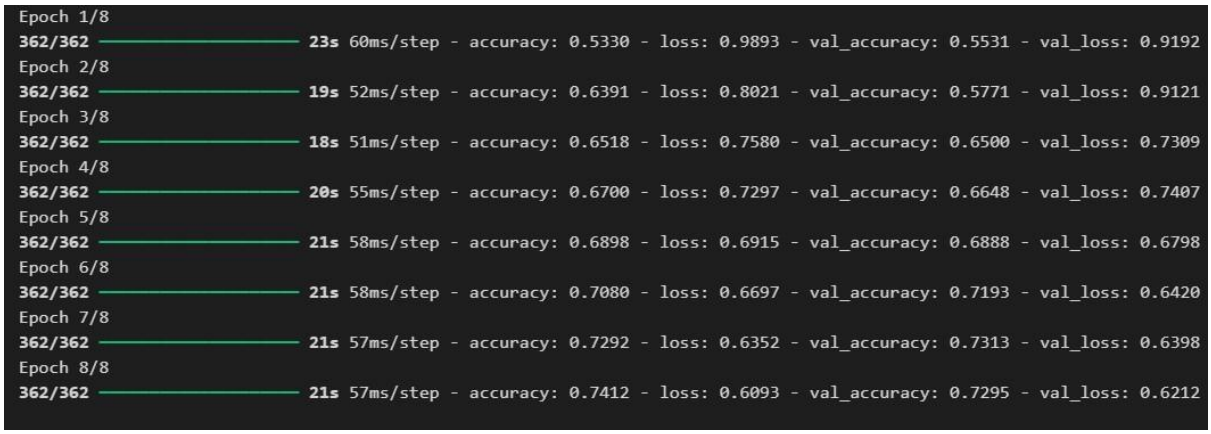

Figure 5 illustrates the GAP results.

Experiment 2: Data Augmentation

Data augmentation was considered a way to increase variation in the training dataset. This involves creating slightly altered versions of the original images, such as applying small rotations or zoom changes. In medical image classification, augmentation must be used carefully so that clinically important features are not distorted.

In this project, data augmentation was shown to be possible for the chest X-ray dataset. Small transformations such as rotation and zoom were applied successfully. This demonstrated that the training images could be modified to create additional variation, which may help improve robustness and reduce overfitting in future experiments.

The purpose of this section was to demonstrate that augmentation could be applied, rather than to fully optimise a new trained model using augmentation.

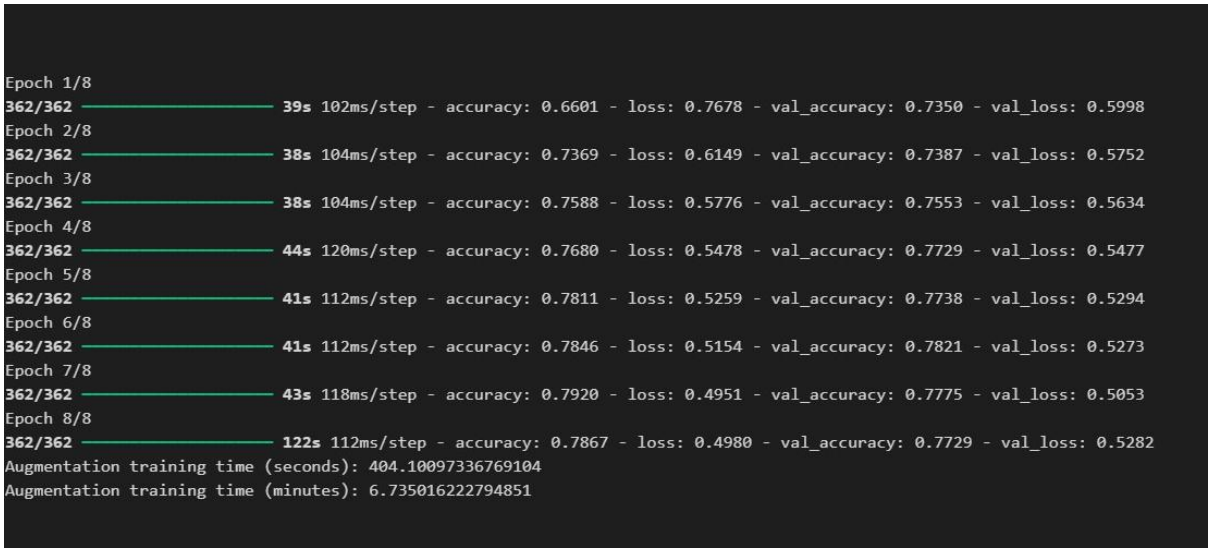

Figure 6 Data augmentation results.

Experiment 3: Transfer Learning

Transfer learning was investigated as an improvement to the baseline CNN. Transfer learning uses a model that has already learned useful image features from a large dataset and adapts it to the current task. In this case, a pretrained model was used with its base layers frozen.

This approach was more computationally demanding than the baseline CNN and took noticeably longer to run. However, the results suggested that transfer learning improved generalisation compared with the baseline model. The gap between training and validation accuracy was smaller, indicating reduced overfitting, while the accuracy remained stronger than in the Global Average Pooling 2D experiment.

This suggested that transfer learning was the most promising improvement method evaluated in this project, although it required more computing time and resources.

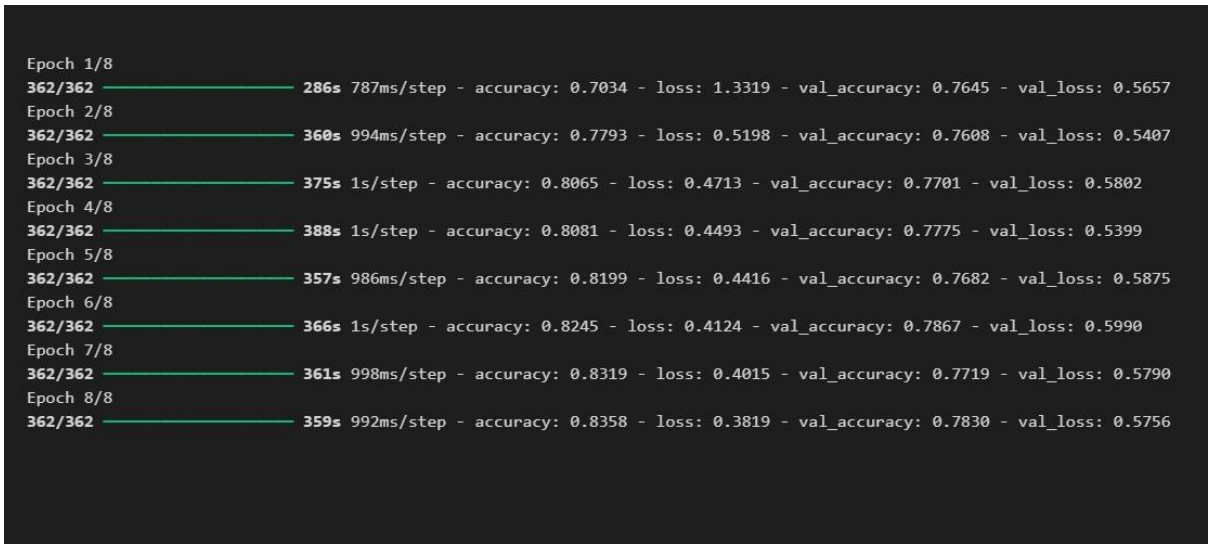

Figure 7Illustrates the Transfer Learning results.

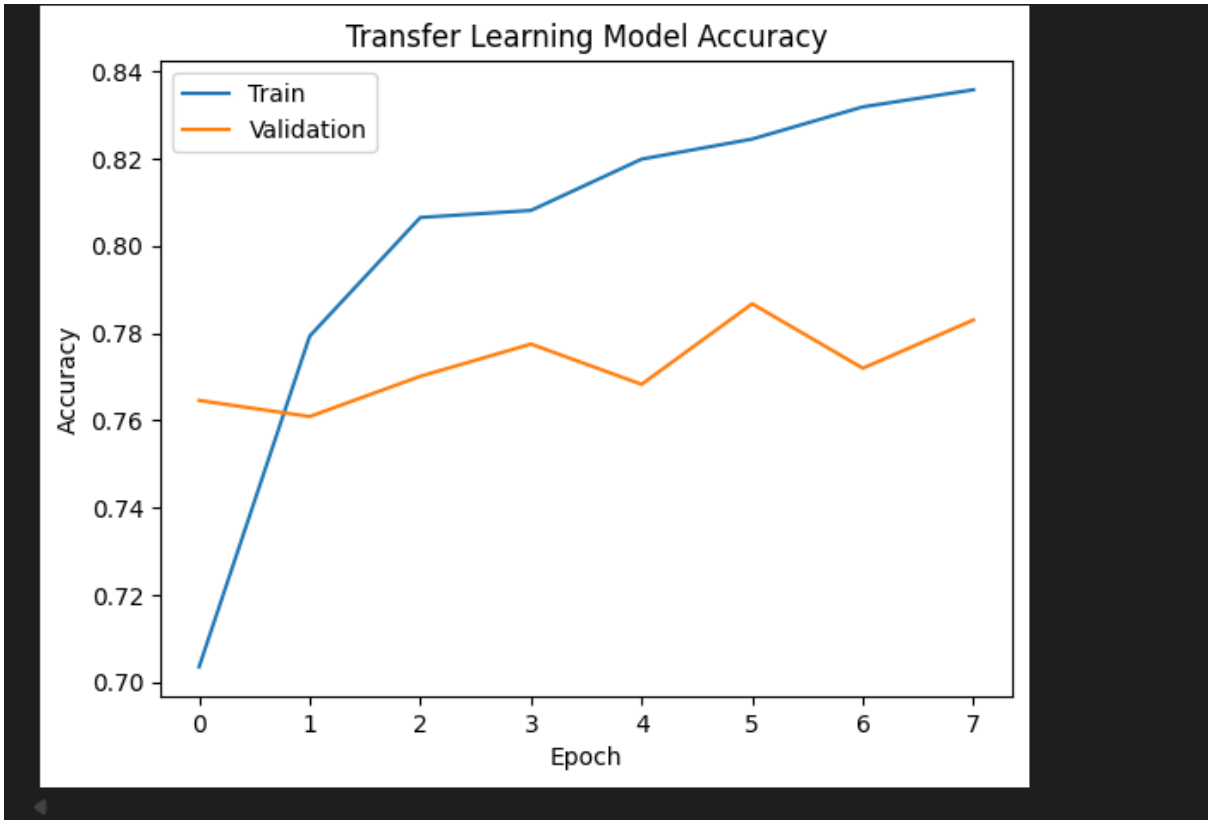

Figure 8 Transfer Learning model Accuracy.

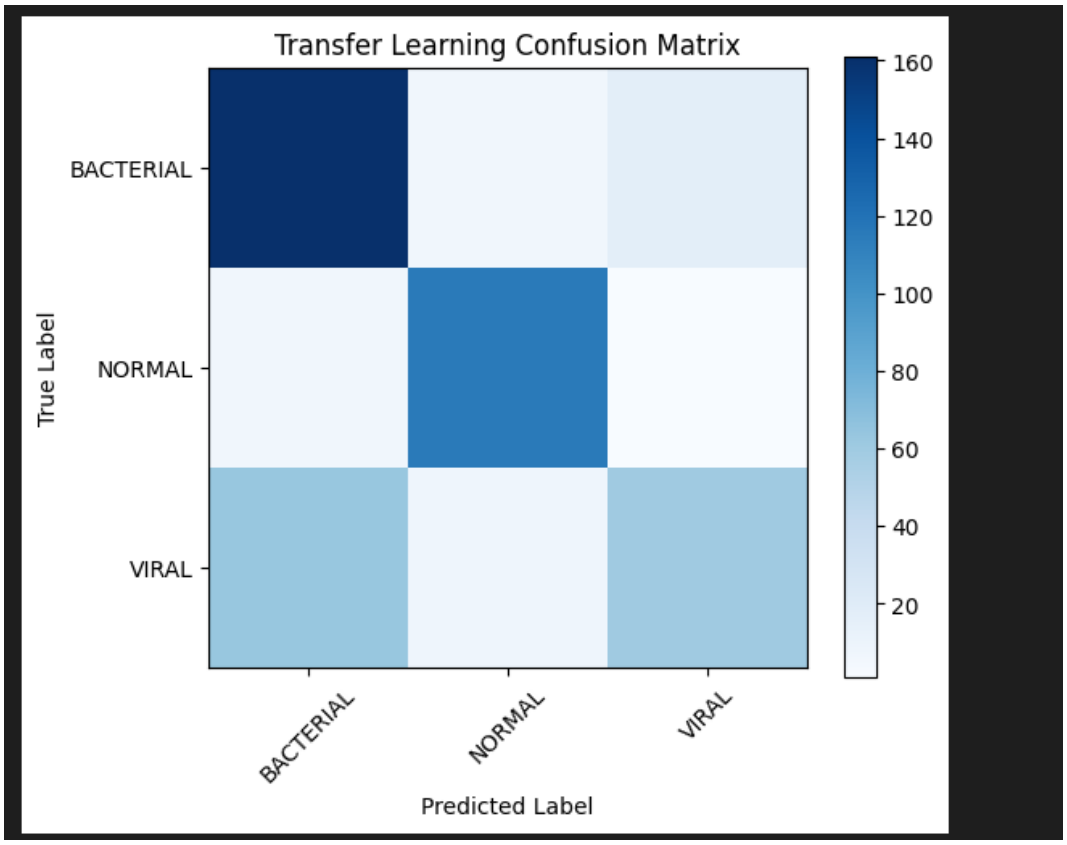

Figure 9 confusion matrix for Transfer learning.

Precision, Recall and F1 Scores

Accuracy alone was not enough to fully evaluate the model, so precision, recall, and F1 score were also considered. These measures provide class-level insight into the model's performance.

Precision measures how many images predicted as a class were correct.
Recall measures how many true images of a class were successfully identified by the model.
F1 score is a balanced measure that combines precision and recall.

These metrics were particularly important in this project because it involved medical image classification. In this setting, recall is especially important for illness-related classes, because missing sick patients is more serious than incorrectly flagging a healthy one.

The classification report and confusion matrix provided a more detailed view of how well the model classified **BACTERIAL**, **NORMAL**, and **VIRAL** images.

	precision	recall	f1-score	support
BACTERIAL	0.75	0.84	0.79	184
NORMAL	0.91	0.94	0.92	122
VIRAL	0.71	0.56	0.63	131
accuracy			0.78	437
macro avg	0.79	0.78	0.78	437
weighted avg	0.78	0.78	0.78	437

Figure 10 Illustrates the classification results of the model.

Improving Detection of Sick Patients

An important aim of this work was to improve the model's ability to identify sick patients correctly. In this project, which meant focusing on the BACTERIAL and VIRAL classes. The main objective was reducing false negatives, where a sick patient is incorrectly classified as normal.

The confusion matrix and class-level recall scores were useful in identifying where the model struggled most. Improving detection of the sick classes would need higher recall for bacterial and viral pneumonia cases, even if this meant accepting some increase in false positives.

The experiments suggested that better generalisation was linked to improved detection performance. Transfer learning offered the strongest potential in this area, while Global Average Pooling 2D reduced overfitting but lowered overall accuracy. Data augmentation may also help by increasing variation in the training data.

Visualising what the CNN Sees

It is possible to inspect what a CNN is focusing on by using explainability methods such as Grad-CAM. Grad-CAM highlights the areas of an image that contributed most strongly to the model's prediction.

In a chest X-ray classification problem, this could be useful in determining whether the CNN was focusing on meaningful lung regions or on irrelevant objects in the image. Although this method was not fully implemented as part of the final experiments, it was identified as an important future step for improving trust and interpretability in the model.

Additional Improvements from Research

Several further improvements were identified during research. These included:

- hyperparameter tuning using Keras Tuner
- adjusting dropout levels
- using batch normalization
- applying class weighting
- further fine-tuning of transfer learning models
- testing learning rate changes

These approaches may improve validation accuracy, reduce overfitting, and strengthen recall for the pneumonia-related classes. They were identified as suitable directions for future work beyond the core experiments completed in this project.

Conclusion

This project evaluated a baseline CNN model for chest X-ray image classification across three classes: BACTERIAL, NORMAL, and VIRAL. The baseline model provided a useful starting point, but it showed signs of overfitting because its training performance was stronger than its validation and test performance.

The experiments showed that replacing Flatten() with GlobalAveragePooling2D() reduced overfitting, although it also lowered accuracy. Data augmentation was successfully

demonstrated and may provide additional robustness in future work. Transfer learning appeared to be the strongest improvement strategy tested, as it reduced overfitting while maintaining stronger validation performance than the other approaches, although it was significantly slower and more computationally demanding.

The project showed that model architecture and training choices had a major effect on performance. The results suggested that transfer learning offered the most promising direction for further improvement, particularly in increasing generalisation and improving detection of pneumonia cases.

https://github.com/Sib26-cyber/Keras_Assignment.git

References

https://www.tensorflow.org/tutorials/keras/keras_tuner

Computer Vision lecture notes

Sklearn, matplotlib, numpy, time, tensorflow and pandas docs